



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-[G01] SYSTEMS INTEGRATION AND ARCHITECTURE 1

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: MediBook: Health Appointment & Records
Integration System

Prepared By: Gyle M. Amihan

Date of Submission: June 30, 2026

Version: 2.0

Table of Contents

1.	Introduction	3
1.1.	Project Title	3
1.2.	System Overview	3
1.3.	Purpose of the System	3
1.4.	Target Users	3
1.5.	Problem Statement	3
1.6.	Scope of the System	4
1.7.	Assumptions and Constraints	4
2.	Stakeholders and Users	4
3.	Functional Requirements	5
3.1.	User Authentication (AUT)	5
3.2.	Appointment Booking & Scheduling (APT)	5
3.3.	Patient Health Records (REC)	5
3.4.	Administrative, System, & Third-Party Integrations (SYS)	5
4.	Non-Functional Requirements	6
5.	Business Rules	6
6.	Requirements Traceability Table	7
7.	Revision History Table	8
8.	System Models (Diagrams)	9
8.1.	Use Case Models	9
8.2.	Entity Relationship Diagram (ERD)	9
8.3.	Activity Diagram	10
8.4.	Sequence Diagram	11

1. Introduction

1.1. Project Title

MediBook: Health Appointment & Records Integration System.

1.2. System Overview

MediBook is an integrated client-server healthcare ecosystem designed to bridge decentralized patient operations and administrative workloads into a unified platform. The system consists of a centralized **Spring Boot REST API** connected to a cloud-hosted **Supabase PostgreSQL database**, serving two distinct frontend applications: a **ReactJS web application** for internal clinic management/administration and an **Android Kotlin mobile application** for patient self-service utilities.

1.3. Purpose of the System

The system transitions small healthcare facilities away from error-prone manual logs. By integrating appointments and fundamental records into a unified digital environment, MediBook eliminates scheduling conflicts, secures historical documentation, and facilitates asynchronous interaction between remote patients and clinical staff.

1.4. Target Users

- **Patients:** Individuals seeking to arrange consultations and check their scheduled queues via their mobile phones.
- **Clinic Staff / Receptionists:** Administrative personnel who handle scheduling logistics, configure general parameters, and maintain triage pipelines via the web panel.
- **Doctors / Healthcare Workers:** Professionals who view active daily pipelines and record diagnostics post-consultation via the web application workspace.
- **System Administrators:** Internal tech coordinators responsible for securely managing roles, creating closed internal employee profiles, and auditing baseline system variables.

1.5. Problem Statement

Small healthcare centers and local barangay clinics in the Philippines regularly operate via manual appointment trackers or localized, disconnected software utilities. This lack of integration leads to scheduling overlaps, structural data loss, and an inability to map patient histories fluidly. Patients face barriers, as they must travel to clinics or make direct phone inquiries to request appointments. Without a centralized architecture, data consistency breaks down, which slows down patient services and limits clinical operations.

1.6. Scope of the System

- **Core Functional Inclusions:** User registration and JWT identity authentication; Administrator control panel for closed internal account provisioning; patient appointment requests and lifecycle tracking ; web-based doctor schedule management and clinical record drafting.
- **External Service Integrations:** Inbound server-to-server connection to a public pharmaceutical/disease taxonomy library (OpenFDA API) for record validation; outbound automated SMTP connection (via Supabase Auth or Mailtrap) for patient email alerts.
- **Explicit Exclusions (Out of Scope):** Video consultations , financial payment processing gates , background carrier-based SMS broadcasts , electronic prescriptions , multi-clinic server synchronization , and native iOS platform clients.

1.7. Assumptions and Constraints

- **A-001:** Users possess persistent internet access while initiating data transactions.
- **A-002:** Patient users operate modern consumer hardware running Android SDK distributions.
- **C-001:** The system must run entirely on free-tier cloud architecture (Render for the Spring Boot Backend API, Netlify for the Frontend Web UI, and Supabase for the database layer).
- **C-002:** All software engineering, end-to-end testing, integration setups, and deployments must be completed within a strict 4-week solo development lifecycle.

2. Stakeholders and Users

- **Patient (User/Stakeholder):** Accesses the Android client to view doctor schedules and submit appointment requests.
- **Clinic Staff (User/Stakeholder):** Acts as a primary workflow manager within the ReactJS web suite to manage operational rosters, adjust booking conflicts, and oversee daily schedules.
- **Doctor (User/Stakeholder):** Uses specialized views within the web dashboard to review their active daily queues and write historical consultation records.
- **System Administrator (User/Stakeholder):** Uses a restricted panel within the web application to securely onboard verified internal employee accounts (STAFF and DOCTOR), locking the application ecosystem from public registration exploits.
- **Course Instructor (External Stakeholder):** Evaluates system design, API endpoint structures, security implementation, and data integrity.

3. Functional Requirements

3.1. User Authentication (AUT)

- **FR-001:** The system shall authenticate users using JSON Web Tokens (JWT) through an encryption pipeline.
- **FR-002:** The system shall grant role-based access levels, restricting backend routes to authorized profiles (PATIENT, STAFF, DOCTOR, ADMIN).

3.2. Appointment Booking & Scheduling (APT)

- **FR-003:** The Android app shall allow authenticated patients to select an active doctor, choose an available date/time slot, and submit booking requests.
- **FR-004:** The web app dashboard shall allow clinic staff to approve, cancel, or modify incoming appointment requests.
- **FR-005:** The web app shall allow clinic staff to generate and configure available doctor calendar time slots.
- **FR-006:** The system backend shall programmatically prevent double-bookings or multiple allocations on identical doctor time slots.

3.3. Patient Health Records (REC)

- **FR-007:** The web app shall allow authorized doctors to create a health record containing a diagnosis and consultation notes upon completing an appointment.
- **FR-008:** The web dashboard shall allow staff and doctors to view patient records, while preventing patients from altering health record content.

3.4. Administrative, System, & Third-Party Integrations (SYS)

- **FR-009:** The web application shall provide an authenticated System Administrator (ADMIN) access to an account management portal to manually provision STAFF and DOCTOR internal profiles.
- **FR-010:** The system backend shall securely initiate server-to-server HTTP REST actions against an external public data registry (OpenFDA API) to dynamically fetch standardized medical classifications when a doctor inputs clinical diagnostic text.
- **FR-011:** The system backend shall communicate with an outbound SMTP mailing gateway (via Supabase or Mailtrap) to automatically compose and distribute plaintext appointment status confirmation updates directly to a patient's registered email address.

4. Non-Functional Requirements

- **NFR-001 (Security):** The system shall encrypt all user passwords using BCrypt hashing before persisting them into the Supabase PostgreSQL database instances.
- **NFR-002 (Performance):** The Spring Boot REST API shall return JSON response data inside a 3-second operational threshold under normal execution loads.
- **NFR-003 (Usability):** The Android layouts shall follow responsive Material Design guidelines for touch gesture layout and navigation.
- **NFR-004 (Reliability):** The database platform transactions shall preserve ACID data integrity constraints across concurrent requests.
- **NFR-005 (Compatibility):** The mobile client must be compatible with Android devices running Android OS version 8.0 (API Level 26) or higher.
- **NFR-006 (Maintainability):** The backend codebase shall implement a distinct layered architecture separating Controllers, Services, and Repositories.

5. Business Rules

- **BR-001 (State Machine Constraints):** All newly submitted appointments default to a PENDING status. They can only transition to CONFIRMED or CANCELLED via clinic staff actions, and to COMPLETED via direct doctor authorization.
- **BR-002 (Account Uniqueness):** No two records within the core authentication schema can share an identical username or email attribute string.
- **BR-003 (Slot Exclusivity):** A doctor profile cannot be allocated overlapping or identical operational scheduling blocks within the database table.
- **BR-004 (Immutable Records):** Once an appointment reaches the COMPLETED state and a corresponding HealthRecord entry is saved, the historical entry cannot be modified or deleted by clinic staff.
- **BR-005 (Closed Production Enrollment):** Public registration pathways on the Android app mobile endpoint shall be restricted to generating accounts with the PATIENT role. Administrative profiles (STAFF, DOCTOR) must be issued manually by an active system ADMIN.

6. Requirements Traceability Table

Requirement ID	Description	Primary Feature	Target UML Model Mappings
FR-001	JWT User Authentication	User Authentication	Use Case Model, Sequence Diagram
FR-002	Role-Based Access Control	User Authentication	Use Case Model, Class Diagram
FR-003	Patient Appointment Submission	Appointment Booking	Use Case Model, Activity Diagram, Sequence Diagram
FR-004	Status Lifecycle Management	Appointment Booking	Use Case Model, Activity Diagram, Sequence Diagram
FR-005	Doctor Schedule Configuration	Schedule Management	Use Case Model, Class Diagram, ERD
FR-006	Conflict Prevention Guardrails	Schedule Management	Activity Diagram, ERD
FR-007	Health Record Creation	Patient Health Records	Use Case Model, Class Diagram, Sequence Diagram
FR-008	Read-Only Access Restrictions	Patient Health Records	Use Case Model, Class Diagram
FR-009	Closed Employee Account Creation	Administrative Panel	Use Case Model, Class Diagram, ERD
FR-010	External Medical Classification Fetch	Inbound Integration	Use Case Model, Sequence Diagram
FR-011	Automated Confirmation Dispatches	Outbound Integration	Use Case Model, Activity Diagram, Sequence Diagram

7. Revision History Table

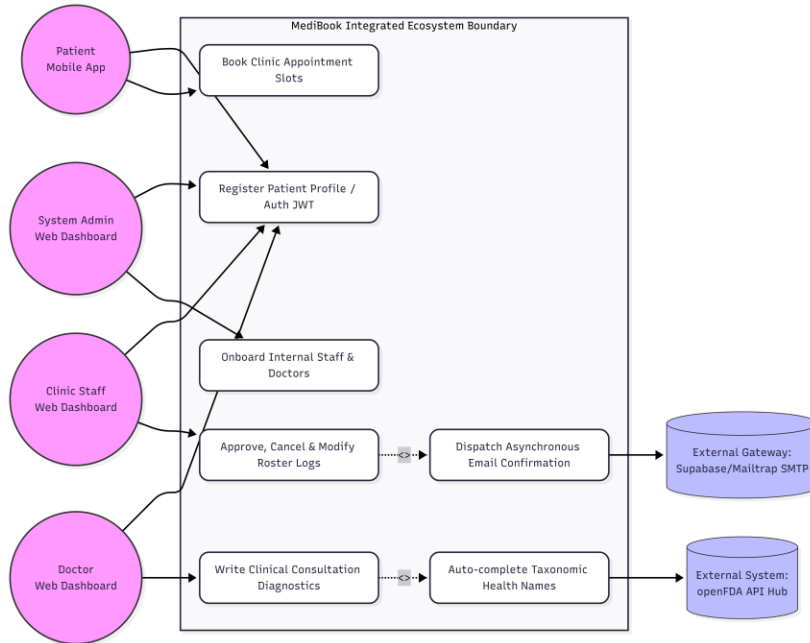
The requirements engineering process was applied to refine the initial scope. Changes optimize the workload for a solo developer by removing structural bottlenecks, establishing explicit data pull mechanics instead of real-time push synchronization, and flattening data entities for cleaner API authentication.

Version	Date	Description of Changes
1.0	June 29, 2026	Initial SRS submission mapping baseline requirements.
2.0	June 30, 2026	Revised: Shifted database architectural layer from MySQL to Supabase Cloud (PostgreSQL Engine) to match deployment layout whiteboard specs. Consolidated redundant functional requirement blocks (FR-009 and FR-010 duplicate loops). Added: System Administrator (ADMIN) parameters, unique ID references (FR-001 through SYS-011), explicit business rules (BR-001 through BR-005), inbound OpenFDA REST API Integration, and outbound SMTP Transactional Mail Integration.

8. System Models (Diagrams)

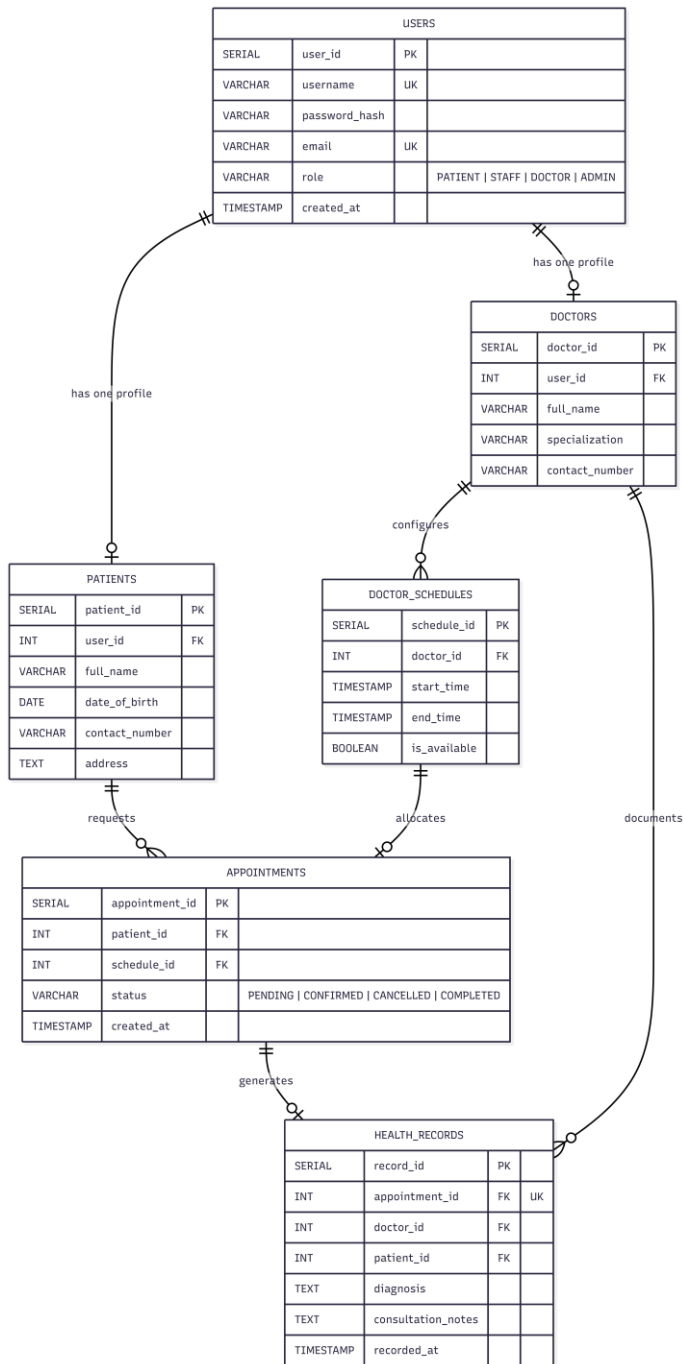
8.1. Use Case Models

This boundary map highlights the target personas interacting with the internal platform functionalities alongside the secondary integrated cloud web service platforms (OpenFDA API and your SMTP Email Delivery System).



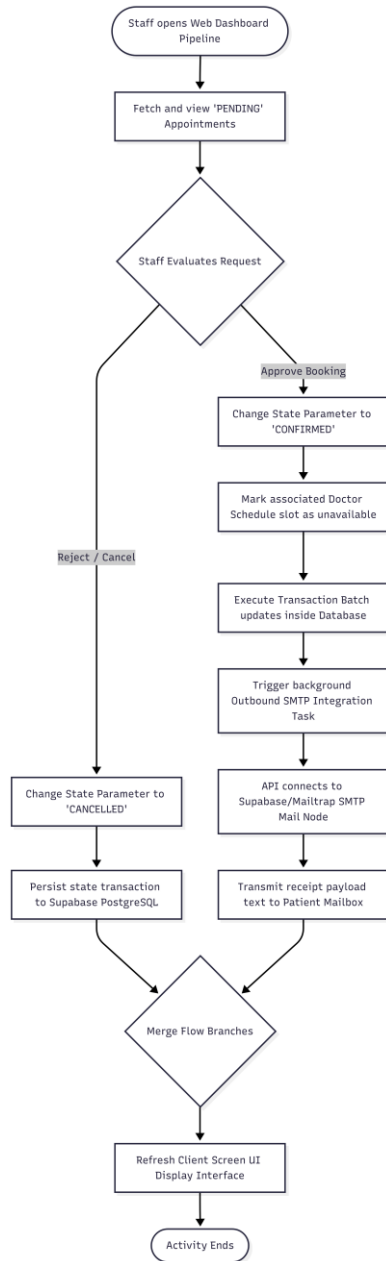
8.2. Entity Relationship Diagram (ERD)

This database layer schema maps directly to your **Supabase PostgreSQL** instance, defining primary keys, operational foreign keys tracking identity hierarchies, enumeration bounds, and relational cardinality rules.



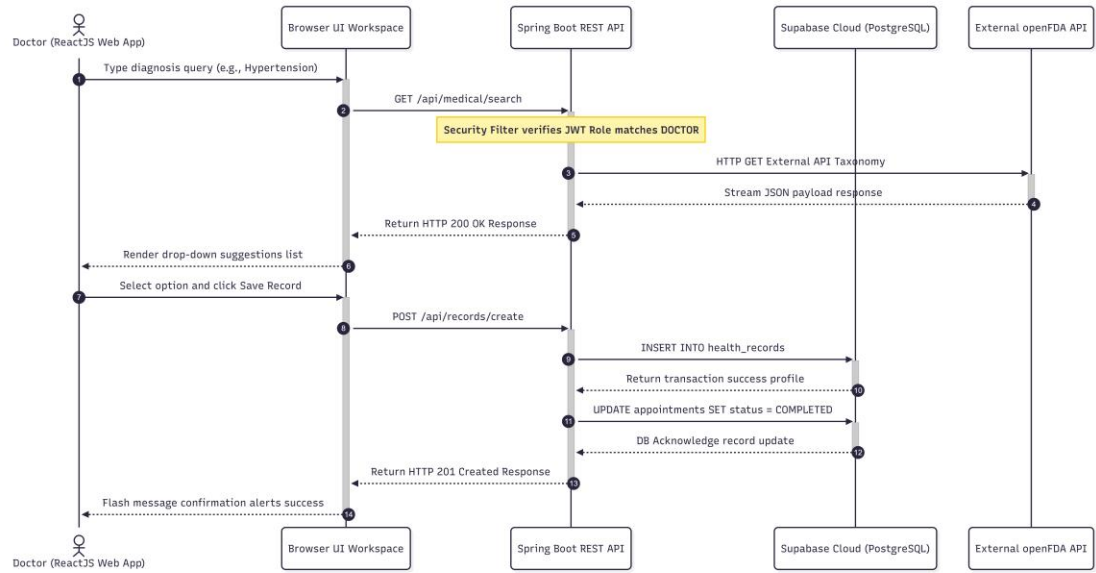
8.3. Activity Diagram

This behavioral blueprint traces the operational workflow sequence governing the **Appointment Approval and Email Delivery Pipeline**, validating status checks and launching third-party cloud data routing loops.



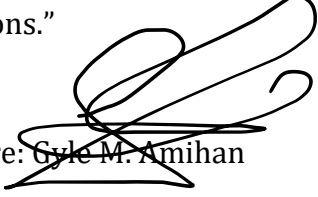
8.4. Sequence Diagram

This structural tracking diagram documents an advanced **Inbound Integration Sequence**: a Doctor drafting consultation logs and triggering an external, server-to-server HTTP REST autocomplete validation lookup against the live openFDA API.



Declaration Statement

"I certify that this finalized SRS and UML models are my own individual work. I understand that copied, duplicated, or AI-generated submissions without proper understanding and revision may be subject to verification and possible deductions."

Signature:  Kyle M. Amihan